Congressional Stock Trade Tracker and Alert System

4+1 Architectural Views

CSCI 375 - Object-Oriented Programming

Fall 2026

Team Members

Caleb Bishop

Eric Dellet

Design Phase

UML architecture prepared before implementation.

The five views describe the planned structure, runtime behavior, code organization, deployment environment, and primary user scenarios for the final project.

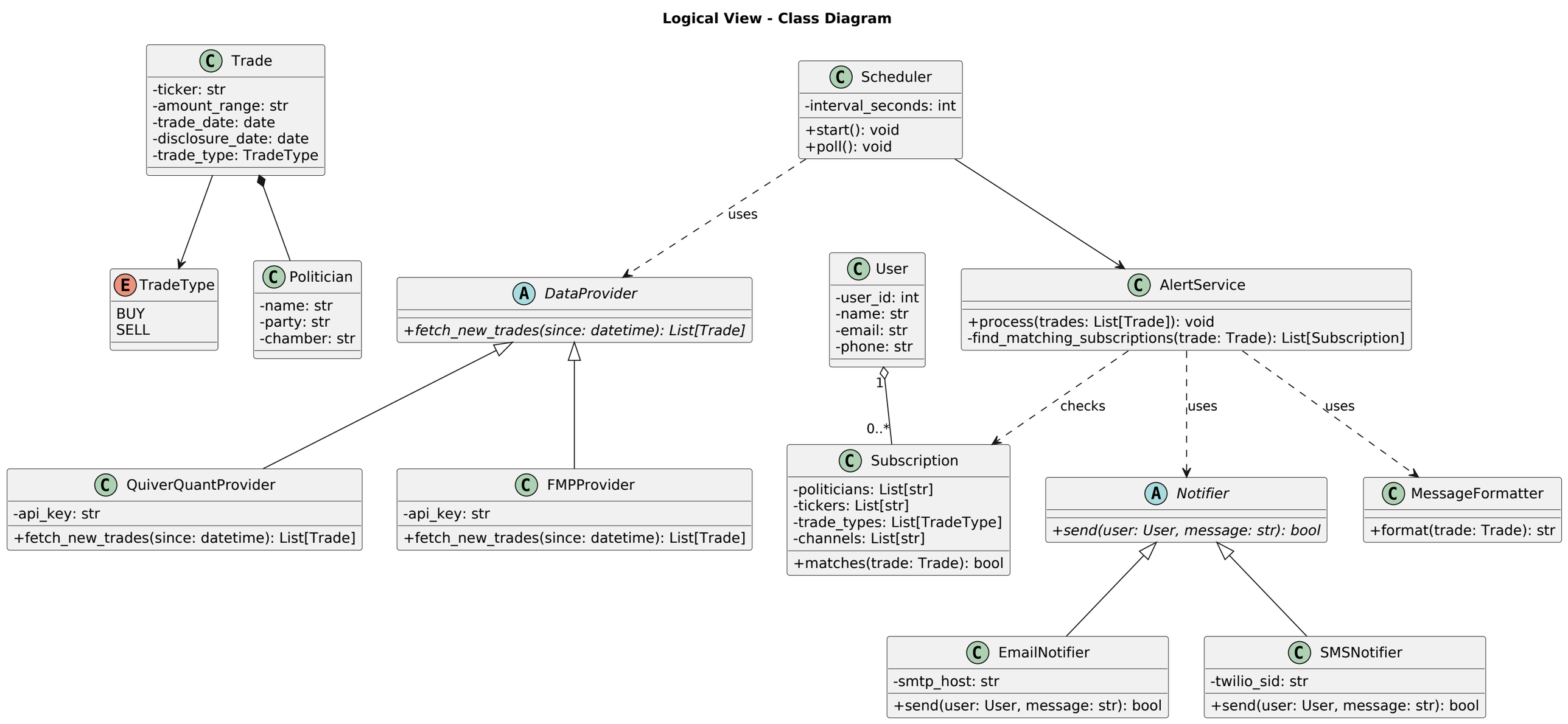
Prepared October 14, 2026

4+1 View Summary

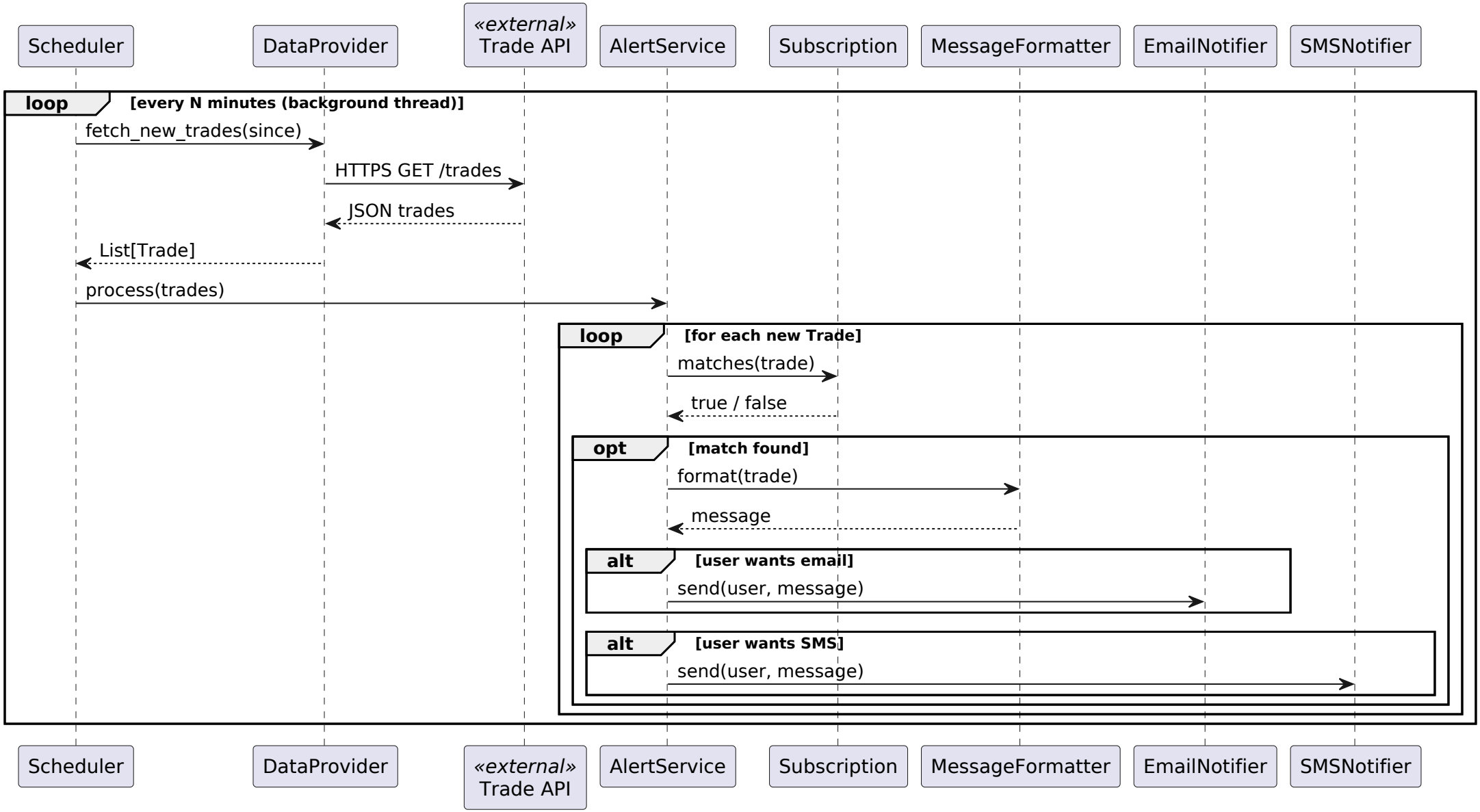
The diagrams in this submission define the planned architecture for the Congressional Stock Trade Tracker. The design emphasizes modular object-oriented components, replaceable data and notification services, and a clear path from trade retrieval to user alerts.

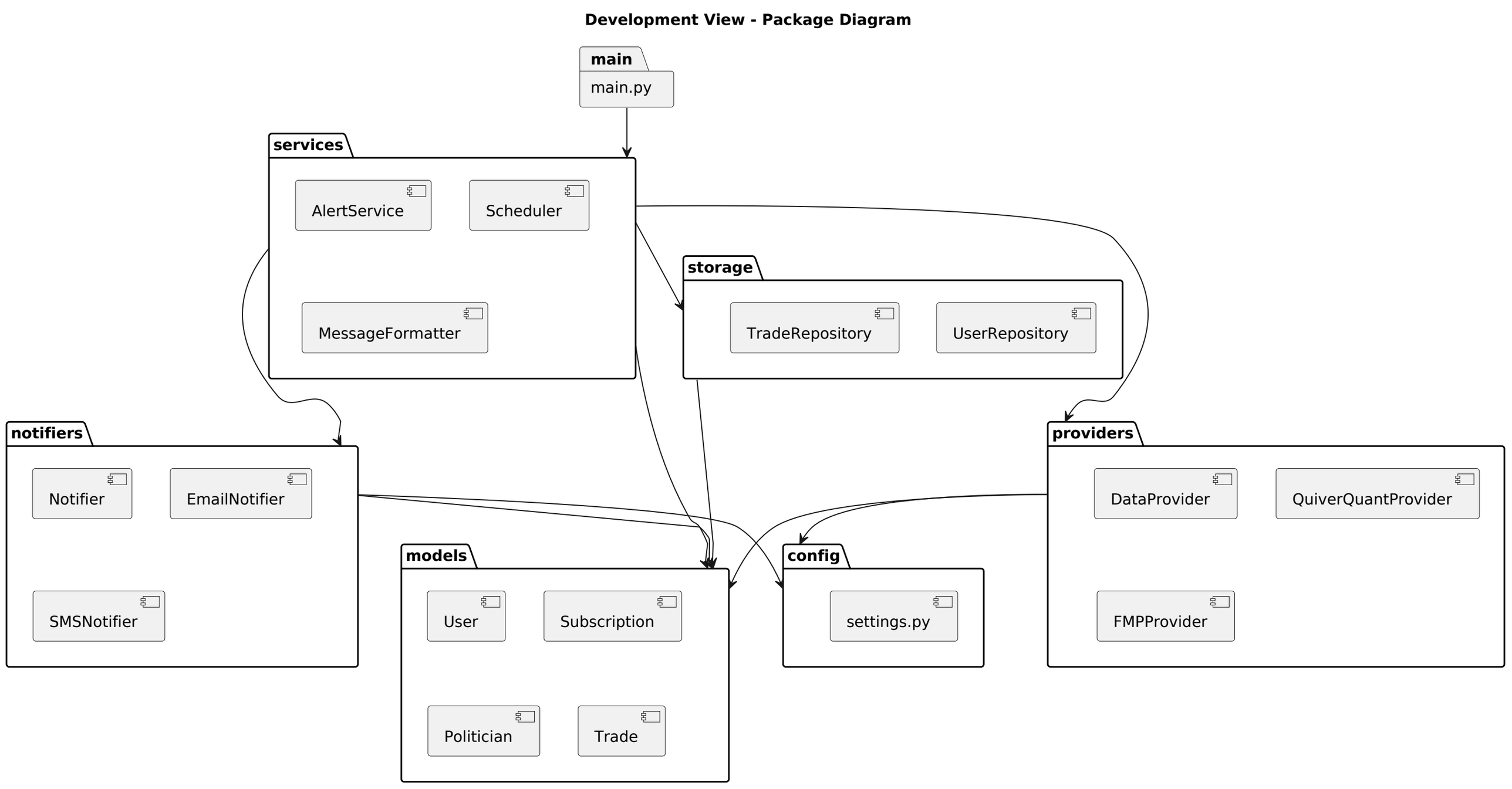
View	UML Diagram	Purpose
1. Logical View	Class Diagram	Defines the main domain, service, provider, and notifier classes and their OOP relationships.
2. Process View	Sequence Diagram	Shows the polling and alert workflow from an external trade API through matching and notification.
3. Development View	Package Diagram	Organizes the planned Python code into models, providers, services, storage, notifiers, config, and main.
4. Physical View	Deployment Diagram	Shows the application server, database, external APIs, notification services, and user device.
+1. Scenarios View	Use Case Diagram	Captures the user and admin goals that connect the other four architectural views.

Note: These diagrams are design artifacts. Specific implementation details may be refined during development while preserving the responsibilities and relationships shown here.

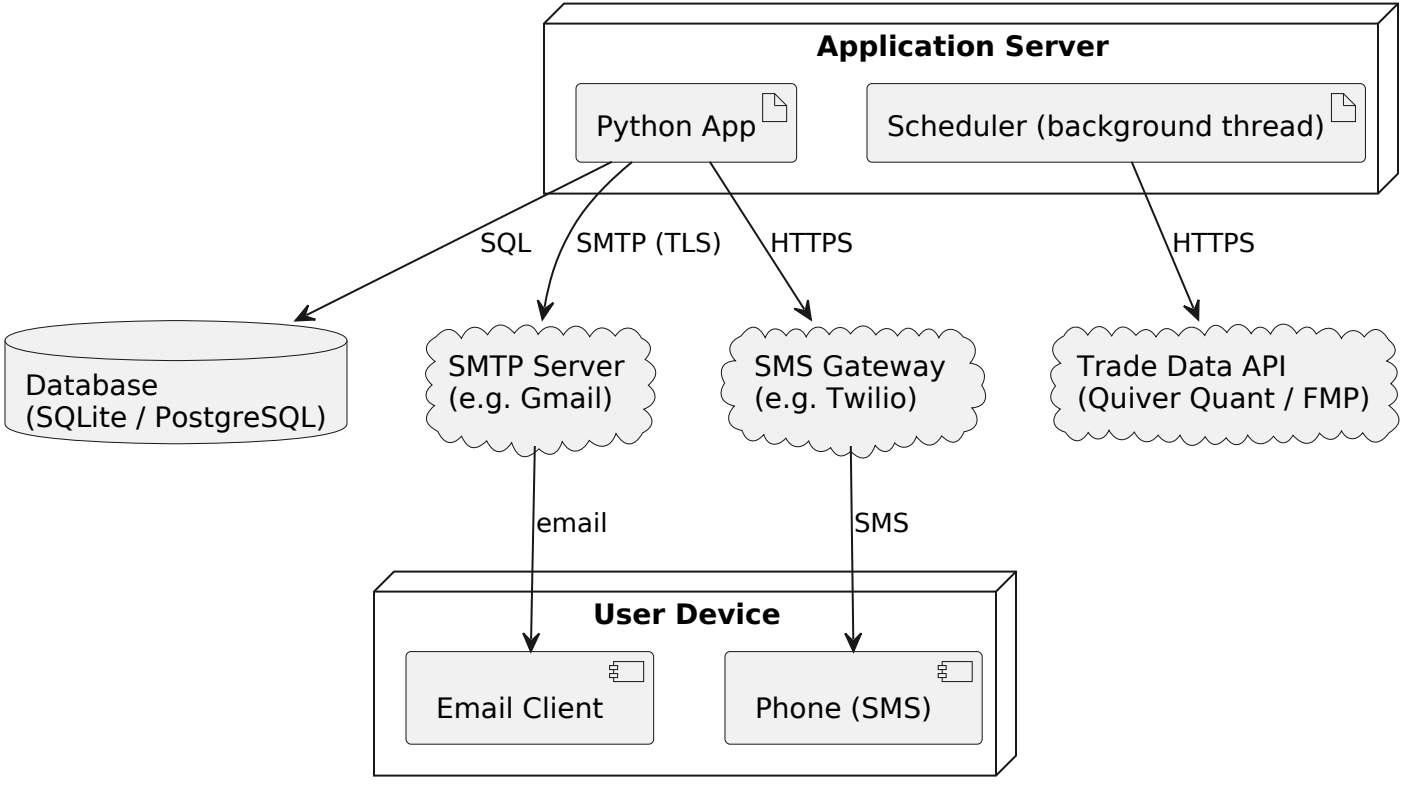

Logical View - Class Diagram
Trade
-ticker: str
-amount_range: str
-trade_date: date
-disclosure_date: date
-trade_type: TradeType
Scheduler
-interval_seconds: int
+start(): void
+poll(): void
uses
TradeType
BUY
SELL
Politician
-name: str
-party: str
-chamber: str
DataProvider
+fetch_new_trades(since: datetime): List[Trade]
User
-user_id: int
-name: str
-email: str
-phone: str
AlertService
+process(trades: List[Trade]): void
-find_matching_subscriptions(trade: Trade): List[Subscription]
1
0..*
checks
uses
uses
QuiverQuantProvider
-api_key: str
+fetch_new_trades(since: datetime): List[Trade]
FMPProvider
-api_key: str
+fetch_new_trades(since: datetime): List[Trade]
Subscription
-politicians: List[str]
-tickers: List[str]
-trade_types: List[TradeType]
-channels: List[str]
+matches(trade: Trade): bool
Notifier
+send(user: User, message: str): bool
MessageFormatter
+format(trade: Trade): str
EmailNotifier
-smtp_host: str
+send(user: User, message: str): bool
SMSNotifier
-twilio_sid: str
+send(user: User, message: str): bool

Process View - Polling and Alert Flow




Development View - Package Diagram
main
main.py
services
AlertService
Scheduler
MessageFormatter
storage
TradeRepository
UserRepository
notifiers
Notifier
EmailNotifier
SMSNotifier
providers
DataProvider
QuiverQuantProvider
FMPProvider
models
User
Subscription
Politician
Trade
config
settings.py

Physical View - Deployment Diagram



Scenarios View - Use Case Diagram

